

Compiling the Rho Calculus to Data-Parallel Hardware through Name-Free Combinators

ANONYMOUS AUTHOR(S)

Process calculi with mobile names are a natural description of concurrent systems and a poor fit for graphics processors. An interpreter for such a calculus matches patterns modulo structural congruence, manages binders, and chases pointers through a store of variably shaped terms; a GPU wants flat arrays of fixed-width records, uniform work across lanes, and bulk-synchronous phases built from sort, scan and join. We close this gap for the rho calculus, a reflective higher-order process calculus in which names are quoted processes, by compiling it into a calculus of *name-free combinators* whose states are multisets of fixed-width tuples of integers.

The compiler runs in two phases. Phase one eliminates the input binder by reflection: a continuation is quoted once and released by a three-atom gate, so the image is linear in the source, where the classical combinator elimination of the asynchronous π -calculus is exponential in nesting depth. Phase two eliminates the auxiliary name binder by running each instance of compiled code at a run-time *address*. We show that the obvious way to do this, erecting apparatus with fixed-arity name constructors, is unsound on a bulk-synchronous machine: concurrently erected instances mix their apparatus in up to all runs. We prove that no fixed-arity construction satisfies the discipline that prevents mixing, and that a single-premise *context-instantiation* combinator does, erecting any instance in two steps. The translation is fully abstract relative to encoded contexts.

On the target, structural congruence is free, pattern matching has moved into the compiler, redex finding is a relational join on a term-sized incidence matrix, and a machine step is a maximal independent set of redexes, which we prove sound with respect to interleaving semantics. We implement the compiler and the machine in Rust with WGSL kernels generated from the rule table, and evaluate on program families whose parallel structure is known. Machine steps track causal depth rather than work: parallel steps stay constant as width grows 64-fold, and a balanced distributor, a one-line change to phase one, reduces the depth of a higher-order benchmark from 266 steps to 20.

CCS Concepts: • **Software and its engineering** → **Compilers**; • **Theory of computation** → **Process calculi**; • **Computing methodologies** → *Parallel algorithms*.

Additional Key Words and Phrases: rho calculus, process calculi, combinators, compilation, GPU, bulk-synchronous parallelism, multiset rewriting, full abstraction

ACM Reference Format:

Anonymous Author(s). 2027. Compiling the Rho Calculus to Data-Parallel Hardware through Name-Free Combinators. *Proc. ACM Program. Lang.* 0, PLDI (September 2027), 19 pages. <https://doi.org/10.1145/nnnnnnnn>.

1 INTRODUCTION

Graphics processors are the most widely deployed parallel machines, and the programming models they reward are narrow. A GPU does well when state is a flat array of fixed-width records, when neighbouring lanes execute the same instructions, and when computation proceeds in bulk-synchronous phases separated by barriers, each phase built from primitives with mature parallel implementations: sort, prefix scan, hash join, sparse matrix product [8, 20, 36]. Programs that can be expressed in these terms inherit decades of tuning. Programs that cannot mostly do not run on GPUs at all.

Concurrent programs written against mobile process calculi are, on their face, in the second class. An interpreter for the π -calculus or one of its relatives stores a soup of terms of varying

shape, reached by pointer. It finds a communication by matching an input against an output modulo structural congruence, and it substitutes under binders, which requires fresh-name management. It resolves competition between redexes by locking or by a central scheduler. Each of these is expensive on a GPU, and together they make the calculus look like the wrong starting point.

This paper argues that it is the right starting point, provided one compiles far enough before running. We take the *rho calculus* [27], a reflective higher-order process calculus in which every name is the quotation $@P$ of a process and every process can be recovered from its quotation by the drop $*x$, and compile it into a calculus of *name-free combinators*: a small, fixed family of atoms, each a head symbol applied to a bounded number of names, with no binders and no pattern variables. The target has four properties that match the four demands of the hardware exactly.

- (1) *States are multisets of fixed-width integer tuples.* Every atom is a head symbol and at most four names, names are interned to integers, and structural congruence of the target is multiset equality, so it costs nothing at run time.
- (2) *Rules are generated and uniform.* There are twenty rules, all generated from a table of twenty-one atom shapes. Each rule is a join of one consumer atom against one to three message atoms on shared names, with no nested matching.
- (3) *A step is a bulk operation.* Redex finding for every rule is a relational join on an atom–name incidence matrix whose size is that of the term, conflict detection is a second product, and a machine step fires a maximal independent set of redexes. We prove that firing an independent set is sound with respect to interleaving semantics, so the number of machine steps is bounded by the causal depth of the computation rather than its length.
- (4) *Reflection becomes an intern table.* The feature that makes the source calculus theoretically demanding — that a name is a program — is, on the machine, a bidirectional hash-consing table. Opening a quoted continuation is a table read followed by a contiguous copy, and building a name is an insert-or-get.

The compiler that produces such terms is the technical heart of the paper, and it turns out to be more delicate than its sequential predecessors suggest.

Binders by reflection. Yoshida [37] showed that the input binder of the asynchronous π -calculus can be eliminated in favour of combinators, keeping name restriction. Her elimination pushes each prefix through its body, splitting at every parallel composition, so the image of a nesting of d inputs has $\Theta(3^d)$ atoms. In the rho calculus a continuation can instead be *quoted once* and stored behind a three-atom gate that releases it when the input fires. The image is linear in the source (§4.1). This is the same move as supercombinator compilation [16] made in a concurrent setting, and it is where reflection first pays for itself.

Freshness at run time, and a race that breaks it. Phase one leaves a name binder, new, around the apparatus of each compiled input, and phase two must remove it. Allocating apparatus names by syntactic position is unsound as soon as compiled code is instantiated twice, which higher-order programs do routinely (§4.2). The fix is to allocate apparatus at run time beneath an *address* each instance receives. But erecting an atom at allocated names on a bulk-synchronous machine is itself a join, and joins at shared channels can draw their inputs from different instances. We measure the consequence — concurrent erection by fixed-arity name constructors mixes the apparatus of distinct instances in 29 of 50 runs with two instances and all 50 runs with eight — and prove it is unavoidable for that construction (Proposition 4.7). A single combinator that instantiates a whole context from one address, in one step with one premise, meets the discipline by construction and erects any instance in two machine steps (§4.3).

Contributions.

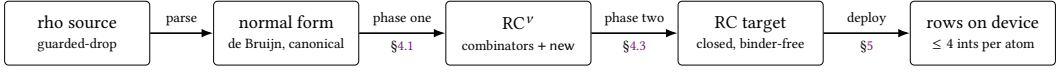


Fig. 1. The compilation pipeline. Phase one removes the input binder by reflection; phase two removes the name binder by run-time addressing; the deployed state is a multiset of fixed-width rows over interned names.

- A repaired name-free combinator calculus for the rho calculus, fit to serve as a data-parallel intermediate representation. We identify the defect that made the published calculus unsound, add the name constructors without which no encoding of the rho calculus exists, and add the context-instantiation combinator *inst* (§3).
- A two-phase compiler from the rho calculus to these combinators, linear in the size of its input, with a proof that the translation is fully abstract relative to encoded contexts and that the relativisation is forced (§4).
- An analysis of instance mixing under concurrent erection: a discipline that excludes it, a proof that fixed-arity constructors cannot meet the discipline, and a single-premise primitive that does (§4.3).
- A bulk-synchronous abstract machine whose step is a maximal independent set of redexes found by joins, with a soundness theorem, and an account of how the expressiveness lattice of the combinators determines which linear-algebraic shape a program's operator admits (§5).
- An implementation, *f1r3comb*, comprising a compiler, a host machine and a GPU machine whose WGS� kernels are generated from the rule table, together with an evaluation of available parallelism, erection schemes and a depth-reducing optimisation of phase one (§6, §7).

What we do not claim. We do not report wall-clock speedups over a tuned sequential interpreter. Our GPU machine runs, and its step traces are identical to those of the host machine, but our evaluation hardware is a software Vulkan driver, and wall-clock numbers from it would be meaningless. We instead measure the quantity on which any speedup from this route depends: the width of compiled programs, meaning the number of rewrites per bulk-synchronous step (§7). Whether real rho workloads are wide is an empirical question our benchmarks do not settle, and we say so.

2 THE RHO CALCULUS

The rho calculus [27] is an asynchronous name-passing calculus in which names are not atoms but quoted processes. Its name is an acronym, for the *reflective higher-order* calculus, and a pun on the letter after π .

Definition 2.1 (Syntax).

$$P, Q ::= \mathbf{0} \mid \text{for}(y \leftarrow x) P \mid x!(P) \mid *x \mid P \mid Q \quad x, y ::= @P$$

An output $x!(P)$ sends a process, and the receiver binds its quotation: in $\text{for}(y \leftarrow x) Q$ the variable y ranges over names, and $*y$ runs the process that y quotes. Structural congruence $\equiv$ makes $(\mid, \mathbf{0})$ a commutative monoid up to α -equivalence. Name equivalence $\equiv_N$ is the least congruence on names with $@*x \equiv_N x$ and $@P \equiv_N @Q$ whenever $P \equiv Q$. Substitution decodes at the drop: $(*y)\{ @Q/y \} = Q$, with the structural clauses otherwise. Reduction is the single rule

$$\frac{x_t \equiv_N x_s}{\text{for}(y \leftarrow x_t) P \mid x_s!(Q) \longrightarrow P\{ @Q/y \}}$$

closed under parallel composition and $\equiv$. We assume source programs are *guarded-drop*: every $*x$ has x bound by an enclosing input, a property preserved by reduction.

```

148  dist :  $d(a, b, c) \mid m(a, v) \longrightarrow m(b, v) \mid m(c, v)$       bindl :  $b_l(a, b) \mid m(a, v) \longrightarrow fw(v, b)$ 
149  kill :  $k(a) \mid m(a, v) \longrightarrow 0$                           bindr :  $b_r(a, b) \mid m(a, v) \longrightarrow fw(b, v)$ 
150  forward :  $fw(a, b) \mid m(a, v) \longrightarrow m(b, v)$         synch :  $s(a, b, c) \mid m(a, v) \longrightarrow fw(b, c)$ 
151  eval :  $e(a) \mid m(a, v) \longrightarrow *v$                     quote :  $fw(a, b) \mid q(a, p) \longrightarrow m(b, @p)$ 
152  makel :  $cons_l(a, b, c) \mid m(a, @p) \mid m(b, @r) \longrightarrow m(c, @(p \mid r))$ 
153  makeA :  $cons_A(a_1, \dots, a_n, f) \mid m(a_1, v_1) \mid \dots \mid m(a_n, v_n) \longrightarrow m(f, @A(v_1, \dots, v_n))$   each base shape  $A$ 
154  inst :  $inst(a, f, C) \mid m(a, v) \longrightarrow m(f, @C[v])$    $C$  a context, holes at names
155

```

Fig. 2. The rules of RC, closed under $\mid$ and the equations, with subjects matched up to $\equiv_N$. The six router rules and quote (class (a)) permute existing names; make and inst (class (b)) build a name; eval (class (c)) opens one.

The calculus has no name restriction. Fresh names are obtained by quotation: the name $@P$ is fresh for any process in which it does not occur, and code can manufacture such names at run time. It is Turing complete; the recursion combinator $D_x = \text{for}(y \leftarrow x) (x!(*y) \mid *y)$ behaves as a replication server when sent itself as a message, since each reception re-sends its own quotation and runs it.

What makes it hard to run in parallel. Four things, which correspond to the four properties listed in §1. A process is a tree of arbitrary shape, so the state of an interpreter is a heap. A redex is found by matching an input against an output whose channels are equal modulo $\equiv_N$, which is a recursive comparison of trees modulo commutativity. Firing it substitutes under the continuation's binders. And two inputs on one channel compete for one output, so firing is a conflict-resolution problem. Tuple-space implementations of the calculus solve each of these sequentially, with locks around the store. The rest of the paper removes each of them by compilation.

3 THE TARGET: NAME-FREE RHO COMBINATORS

The target calculus RC has two sorts, names N and processes P , and is presented as an operational theory: formation rules, equations, and named rewrite rules.

3.1 Atoms and rules

Processes are finite parallel compositions of *atoms*. The base atoms are

$$m(a, v), d(a, b, c), k(a), fw(a, b), b_l(a, b), b_r(a, b), s(a, b, c), e(a), q(a, p)$$

with all arguments names except the payload p of q . An atom $m(a, v)$ is a message carrying the name v on the channel a . The six *routers* d, k, fw, b_l, b_r, s consume a message and redirect, duplicate or discard it. The *destructor* e consumes a message and runs the process its payload quotes. The *store* $q(a, p)$ holds a process that only a forwarder can release. The equations are those of a commutative monoid on $\mid$, together with $@*a = a$ and $*@p = p$; the last makes quote and drop inverse, so a message payload may be a name, and every argument of every atom other than a store is a name. The rules are in Figure 2.

3.2 Three repairs to the published calculus

The combinators descend from an unpublished draft [1] that proposed them as a binder-free target for the rho calculus. Three changes are needed before they can serve as one, and each matters to the machine.

The drop is not the guard. The draft wrote the destructor and the drop with one symbol, and kept $*@p = p$. Since every name is $@Q$ for some Q , the destructor at $@Q$ is then equal to Q itself, so for all P, Q

$$Q \mid m(@Q, @P) \longrightarrow P,$$

and any running process can be annihilated by anyone who knows its quotation. Keeping e and $*-$ as distinct constructors repairs this and leaves the equation available. We use it: with $*@p = p$ a payload is a name, hence an integer on the machine, and every atom is a fixed-width row.

Names must be built as well as opened. Without the constructors, no rule produces a name that was not already present in the redex, hereditarily. Rho reduction does so constantly: substituting into $@(*y \mid R)$ yields $@(Q \mid R)$, a name in neither participant. So no constructor-free combinator term is equivalent to, for instance, $\text{for}(y \leftarrow a) @(*y \mid *y)!(0)$, and the draft's calculus cannot encode the rho calculus at all. The family cons_A , one member per atom shape, restores the missing capability; cons_A assembles the quotation of an A -node from the quotations of its children (§5.2).

Instantiation of a context. The combinator inst builds, from one received name v , the quotation of a whole context C with v in every hole. It is derivable from the constructor family by chaining, so it adds no expressive power to the calculus as a term algebra. It is nevertheless primitive in our target, for a reason that only appears on a parallel machine and that §4.3 makes precise.

3.3 Rule classes

The rules fall into three classes by what they ask of the machine. *Class (a):* dist , kill , forward , bind_l , bind_r , synch and quote produce a fixed multiset of atoms over names in the premise; on the machine they are field copies. *Class (b):* the constructors and inst produce one message whose payload is a new name, obtained by hash-consing a term built from premise names. *Class (c):* eval is the only rule whose conclusion has a size that depends on the value of a name; on the machine it is a table read. The distinction recurs in §5.2, where it separates what can be precomputed from what must be generated lazily.

4 COMPILATION

The translation removes two binders in turn. Phase one compiles into $\text{RC}^\vee$, the combinators extended with a name binder $(\text{new } z)P$, much as Yoshida [37] compiles the π -calculus into combinators that keep new . Phase two removes new . The split is not a convenience: §4.2 shows that a single-phase translation which invents apparatus names by syntactic position is unsound.

4.1 Phase one: eliminating the input binder by reflection

An input $\text{for}(y \leftarrow x)P$ must wait for a message on x , then run P with the received name in place of y . The combinators have no binder, so both halves need a construction: something that holds P until the message arrives, and something that delivers the received name to each place in P that uses it.

Definition 4.1 (Gate and distributor). For fresh b, c put $G(p, S) := (\text{new } b, c) (s(p, b, c) \mid q(b, S) \mid e(c))$. A *distributor* $D(x; p_0, \dots, p_k)$ is any binary tree of k atoms d , with fresh internal channels, whose root consumes at x and whose leaves are $p_0, \dots, p_k$.

LEMMA 4.2 (GATE). $G(p, S)$ has no reduction and is not a message. For any v ,

$$G(p, S) \mid m(p, v) \longrightarrow \text{fw}(b, c) \mid q(b, S) \mid e(c) \longrightarrow m(c, @S) \mid e(c) \longrightarrow S,$$

and these are the only reductions available. $D(x; \vec{p}) \mid m(x, v)$ reduces in k steps to $m(p_0, v) \mid \dots \mid m(p_k, v)$, and in h bulk-synchronous steps, where h is the height of the tree.

	occurrence of y in the body	rewritten in the stored body	link
(o1)	input subject, $\text{for}(z \leftarrow y) S$	subject becomes c_i	$b_l(p_i, c_i)$
(o2)	output subject, $y!(S)$	subject becomes c_i	$b_r(p_i, c_i)$
(o3)	forwarded name, $v!(*y)$	atom deleted	$\text{fw}(p_i, \llbracket v \rrbracket)$
(o4)	drop, $*y$	becomes $e(c_i)$	$\text{fw}(p_i, c_i)$
(o5)	inside $@S, y \in \text{fn}(S)$	subject becomes c_i	chain rebuilding $@ \llbracket S \rrbracket$

Table 1. Occurrence kinds and their links. When several kinds meet in one atom, the atom is deleted and the links are emitted together.

The gate stores the continuation as a quoted process inside q , which only a forwarder can read, and s produces that forwarder when the trigger arrives. The continuation is never rewritten, only stored and released. Yoshida [37, rules (I), (V), (VI)], lacking reflection, must instead delay each atom of the continuation behind its own synchroniser and split at every parallel composition.

The distributor supplies copies of the received name. Occurrences of the bound name y in the body fall into five kinds (Table 1). For the i -th occurrence the translation allocates a proxy c_i , rewrites the occurrence inside the stored body, and emits a *link* outside the gate that connects the distributor's i -th leaf to the proxy.

Definition 4.3 (Phase one). On names, $\llbracket @P \rrbracket := @ \llbracket P \rrbracket$. On processes, $\llbracket 0 \rrbracket = 0$, $\llbracket P \mid Q \rrbracket = \llbracket P \rrbracket \mid \llbracket Q \rrbracket$, $\llbracket x!(P) \rrbracket = m(\llbracket x \rrbracket, @ \llbracket P \rrbracket)$, and

$$\llbracket \text{for}(y \leftarrow x) P \rrbracket = (\text{new } \vec{p}, \vec{c}, \vec{q}) (D(\llbracket x \rrbracket; p_0, \dots, p_k) \mid G(p_0, B) \mid L_1 \mid \dots \mid L_k)$$

where $o_1, \dots, o_k$ enumerate the free occurrences of y in P , L_i is the link of o_i , and B is $\llbracket P \rrbracket$ with each occurrence rewritten as in Table 1.

Leaf p_0 triggers the gate; the others feed the links, which have already fired, or are waiting at their proxies, by the time the body runs. Kind (o4) is where the source's decoding substitution $(*)\{ @Q/y \} = Q$ and the target's eval rule meet: the link forwards the received name to c_i , and the $e(c_i)$ left in the body opens it. Kind (o5) is the only one that constructs a name, and the chain uses one constructor per node on the path from the occurrence to the root of S .

PROPOSITION 4.4 (SIZE). $|\llbracket P \rrbracket| = O(|P|)$, counting atoms. For $P_d = \text{for}(x_1 \leftarrow a) \dots \text{for}(x_d \leftarrow a) 0$, $|\llbracket P_d \rrbracket| = 3d + 1$, while Yoshida's elimination yields $(3^{d-1} + 1)/2$ atoms.

PROOF SKETCH. An input contributes k distributor atoms, three gate atoms and k links, where k is the number of occurrences of its bound name, and its body is translated once and stored once. Summing k over binders counts each occurrence once, and an (o5) chain is linear in the quoted subterm. For P_d , no prefix has an occurrence. Yoshida's count satisfies $T(1) = 1$, $T(j+1) = 3T(j) - 1$: rule (I) inserts $n - 1$ duplicators for a body of n atoms and rule (VI) doubles each atom. $\square$

The distributor can be any binary tree of d atoms, and the original formulation used a left comb, whose height is k . Since every copy of the received name must pass through the tree before the corresponding occurrence can act, the height is on the critical path of every instance. A balanced tree has height $\lceil \log_2(k+1) \rceil$ with the same number of atoms. The choice is invisible to the correctness argument, which uses only Lemma 4.2's conclusion, and it matters a great deal on a bulk-synchronous machine (§7.3).

4.2 Why positional apparatus is unsound

Phase one's output still binds names. A tempting shortcut is to replace each bound name by a name computed from its syntactic position, which is fresh for the program because it quotes the

position. The following program shows why that fails. Let $R = \text{for}(y \leftarrow a) y!(*y)$, which sends on the received channel the process that channel quotes, and let

$$W = \text{for}(z \leftarrow b) (*z \mid *z) \mid b!(R) \mid a!(X) \mid a!(Y).$$

The communication on b runs two copies of R , which in the source are α -distinct. One takes $@X$, the other $@Y$, and $W \longrightarrow^* @X!(X) \mid @Y!(Y)$.

R 's single occurrence of y is both an output subject and a forwarded name, kinds (o2) and (o3), so its two links share one proxy c . Under positional naming, both copies of R are instantiated from the same quoted text and carry the same c . After the links fire, the soup contains

$$\text{fw}(c, [[@X]]) \mid \text{fw}(c, [[@Y]]) \mid \text{m}(c, [[@X]]) \mid \text{m}(c, [[@Y]])$$

and nothing pairs a forwarder with the message from its own copy. A run can produce $@X!(Y) \mid @Y!(X)$, which an observer listening on $@X$ distinguishes from the source. On our implementation the positional scheme produces this crossed result on 49 of 100 random schedules of W (§7.1). The underlying invariant is the distinction between *occurrences*, which positional naming separates, and run-time *instances*, which it does not. A binder separates both, because two instances of $(\text{new } c)P$ are α -distinct. Phase one keeps the binder for that reason, and phase two must eliminate it without giving the property back.

4.3 Phase two: eliminating the name binder by run-time addressing

Phase two runs every instance of compiled code at an *address*, a name it receives at run time, and derives its private names from that address. We call a *unit* any piece of phase-one output whose own level binds names: the top level, the body stored in a gate, and any quoted process that contains an input.

Addresses. Fix two static constant names and write $\sigma \cdot w$ for the name obtained from σ by the path $w \in \{0, 1\}^*$, where each bit applies one of two injective name constructors with disjoint images, $L\sigma = @m(\sigma, @0)$ and $R\sigma = @m(\sigma, K)$ for a fixed $K \neq @0$. Distinct paths give distinct names, and if neither of σ, τ is a prefix of the other their subtrees are disjoint. The j -th bound name of a unit is assigned the leaf at path $\gamma(j)$, the Elias gamma code of j [11]. Gamma codes are prefix-free, so the leaves of one unit are pairwise incomparable and every spare address a unit hands to a child owns a subtree disjoint from its siblings'.

Erection. Let C_U be the context obtained from unit U 's phase-one image by replacing each bound name z_j with $\square \cdot \gamma(j)$, a path beneath the hole. The phase-two image of U is

$$[[U]]^\# := A_U \mid \text{inst}(r^*, f^*, C_U) \mid e(f^*)$$

where A_U collects the atoms of U that mention none of its bound names, and r^*, f^* are static channels. An address σ arriving at r^* fires inst , which delivers $@C_U[\sigma]$ at f^* , and $e(f^*)$ opens it. The instance is erected in two steps, whatever its size, and all its private names are leaves beneath σ .

Supplying addresses. An address must be available whenever code with its own inputs is about to run, which happens at exactly two kinds of site: a gate releasing a body that contains an input, and a drop $*y$, which may run code received as a message. Phase one emits, for every such site, a fresh a with $m(r^*, a)$ alongside the site; after phase two, a is itself a leaf of the enclosing instance's address, so each run-time instantiation receives an address posted by the event that caused it. Root addresses for deployment are chosen larger than every name in the image and pairwise prefix-incomparable.

LEMMA 4.5 (FRESHNESS). *If the root address exceeds every $\equiv_N$ -normal name of the image, then no allocated leaf occurs in the image, and no name constructed at run time by an (o5) chain is a leaf.*

PROOF SKETCH. Leaf construction strictly increases size on normal forms, which gives the first claim. An (o5) chain builds only from names received on source channels, and addresses never enter the payload of a source message, so no constructed name mentions an address. The second clause is an obligation on the compiler, and f1r3comb checks it after compilation. $\square$

Nested units need indexed holes. A unit nested in another sits inside its parent's context. If both use one hole marker, filling the parent with its address also fills the child's hole, and every instance of the child inherits the parent's names. This happens whenever an inner input mentions an outer bound name, as in $\text{for}(y \leftarrow a) \text{for}(z \leftarrow b) y!(*z)$. Holes therefore carry a de Bruijn index over enclosing contexts [9]: $\square_0$ is the innermost context's hole, and inst fills its own hole while shifting deeper ones.

4.4 Concurrent erection, and why fixed-arity constructors fail

Why is inst primitive, when the constructor family can build the same names? Because on a machine that fires many redexes at once, how a name is built matters as much as which name is built.

The natural erection without inst uses the family directly: to erect an atom $A(z_1, \dots, z_m)$ of a unit at allocated leaves $t_1, \dots, t_m$, emit $\text{cons}_A(t_1, \dots, t_m, f_A) \mid e(f_A)$ and let an allocation gadget deliver each leaf as a message on a statically named channel. Every instance of the unit shares those static channels. With two instances live at σ and τ , the constructor for the distributor atom $d(a, p_0, q_1)$ can take p_0 from σ and q_1 from τ and erect $d(a, \sigma \cdot 00, \tau \cdot 01)$. From that point on, values cross between instances, and the failure of §4.2 returns. We measure how often in §7.4: often, and more often under maximal-progress scheduling than under sequential choice, because a bulk-synchronous step fires every enabled constructor while both instances' leaves are present.

The condition that prevents this can be stated locally.

Definition 4.6 (Single-address discipline). Fix an instance at address σ . A redex *joins at a static channel* if one of the channels it consumes is static and two of its premises contribute occurrences of σ to the atoms it produces. A compiled image satisfies the discipline if no reachable redex joins at a static channel.

PROPOSITION 4.7 (ADDRESS MULTIPLICITY). *Under the discipline, and with the rules of Figure 2 other than inst, no reachable atom contains two occurrences of an address σ . Hence no atom mentioning two leaves beneath σ is ever erected — in particular, neither the distributor atom $d(q_1, p_1, q_2)$ nor the gate's $s(p_0, b, c)$.*

PROOF. By induction on reduction. Initially the image is closed and the address arrives as $m(r^*, \sigma)$, one occurrence. Every rule other than inst builds its produced atoms from the consumer's arguments, the payloads of its consumed messages, and static names; for eval, the components of a single payload. A produced atom with two occurrences of σ must draw one from each of two premises. At least one of those is a payload carrying σ , consumed at a channel that, by the induction hypothesis applied to the message atom, is free of σ and hence static. The redex then joins at a static channel, which the discipline excludes. A leaf beneath σ contains exactly one occurrence of σ . $\square$

A fixed-arity constructor places each input once. Building an atom that mentions σ twice from one copy of σ requires duplicating it and joining the copies again, and the join is the forbidden one. So the difficulty lies not in the arity of the atoms but in the premise count of the rule: a two-premise rule can pair premises from different instances, since matching is on subject names alone, while a

one-premise rule cannot mispair anything. *inst* has one premise and places its input at every hole, so its image satisfies the discipline by construction, and addresses at r^* are then interchangeable.

LEMMA 4.8 (PERMUTATION). *Let instances run at pairwise prefix-incomparable addresses in an image erected by *inst*. Every reachable configuration is one in which each instance's atoms mention leaves of a single address, and reachable configurations differ from the uncrossed ones by a permutation of addresses among instances.*

We note one alternative. A *curried* constructor $\text{cons}_A^*[\vec{w}](t, f) \mid m(t, \sigma) \longrightarrow m(f, @A(\sigma \cdot w_1, \dots, \sigma \cdot w_n))$ also has one premise and keeps the signature fixed-arity. It erects one atom per rule, so a unit costs a fan-out tree of copies of σ plus one constructor per atom, and it cannot erect a stored body with two atoms that mention the unit's names without a join at a derived channel. We implement it as an option and compare the two in §7.4; the device targets *inst* only.

4.5 Correctness

We state correctness as full abstraction for an observational equivalence defined by context-labelled transitions [23, 33]. For a calculus with contexts $\mathcal{C}$ and a class of observers $\mathcal{D} \subseteq \mathcal{C}$, write $P \xrightarrow{C} P'$ when $C[P] \longrightarrow P'$, and let $\approx_{\mathcal{D}}$ be the largest symmetric relation such that $P \approx_{\mathcal{D}} Q$ and $P \xrightarrow{C} P'$ with $C \in \mathcal{D}$ imply $C[Q] \longrightarrow^* Q'$ with $P' \approx_{\mathcal{D}} Q'$. On the source the observers are all rho contexts; on the target, the images $[[\mathcal{C}]]$ of rho contexts.

THEOREM 4.9 (FULL ABSTRACTION RELATIVE TO ENCODED CONTEXTS). *For all guarded-drop rho processes P, Q ,*

$$P \approx_{\mathcal{C}} Q \iff [[P]]^{\#} \approx_{[[\mathcal{C}]]} [[Q]]^{\#}.$$

PROOF SKETCH. The argument follows Anonymous [2]. The translation is compositional on the nose, $[[C[P]]] = [[C]] [[P]]$. An erected input is inert and contains no message. Within one instance every allocated name is the subject of at most one atom and carries at most one message, so each link is a one-shot forwarder, and a term with finitely many links added is bisimilar to one with the links discharged (*link expansion*). The key lemma is *release*: an erected input, given a message on its channel, reduces in a number of steps linear in k to a term that is a link expansion of the translation of the substituted body, and no other reduction sequence is available up to the order of independent steps. Kinds (o1)–(o3) contribute forwarders, (o4) contributes exactly the decoding substitution, and (o5) rebuilds the substituted quotation node by node. *Address independence* – instances at incomparable addresses are bisimilar under the bijection $\sigma \cdot w \mapsto \tau \cdot w$, because an encoded context never names an allocated leaf – together with Lemma 4.8 absorbs the permutation of addresses at r^* . The forward direction builds a bisimulation from *release* and *link expansion*; the backward direction reads *release* left to right, so a distinguishing source context yields a distinguishing encoded context. $\square$

PROPOSITION 4.10 (THE RELATIVISATION IS FORCED). *There are $P \approx_{\mathcal{C}} Q$ and a combinator context $C \notin [[\mathcal{C}]]$ with $C[[P]]^{\#} \not\approx C[[Q]]^{\#}$.*

PROOF. An arbitrary combinator context can name the gate channel b of a translated input and install $\text{fw}(b, u)$, extracting the stored continuation at an observable u (Lemma 4.2). Two source terms differing only in a continuation that is never reached are then distinguished. No encoded context installs such a forwarder. $\square$

The same shape of result holds for the fully abstract encoding of the π -calculus in the rho calculus [3, 25], for the same reason: the target manufactures names the source lacks, and an unrestricted observer can use them. It is also where the store q earns its place. With the continuation

held as a message $m(b, @B)$ rather than a store, no action by the observer would be needed; the store is the continuation made quiet.

5 A BULK-SYNCHRONOUS MACHINE FOR THE COMBINATORS

5.1 States are multisets of rows

A *species* is a head symbol applied to a tuple of names, modulo $\equiv_N$. A target state is a finite multiset of species: the multiset of top-level atoms of the state's drop-free normal form is a complete invariant for structural congruence [4], so $P \equiv Q$ iff their multisets agree. The monoid laws of $|$ are therefore the algebra of the data structure, and $*@p = p$ and $@*a = a$ are discharged once, when names are interned. Matching modulo structural congruence, a recurring cost for a direct interpreter, costs nothing here.

Names are processes, and processes are multisets of species, so the index set of the representation indexes itself: $N \cong M_{\text{fin}}(S)$ with $S = \coprod_A N^{\text{ar}(A)}$. This is reflection seen from the data structure, and it is why no finite matrix can be written down in advance. The machine realises N by an *intern table*, a bijection between integers and canonical component multisets with two directions: quote, hash-consing [12], used by class (b) rules; and drop, a table read, used by class (c). Interning is structural: a name is stored as a row whose components are the indices of its children, so building a leaf one level beneath an address of any depth is one new entry, and no hash of a full encoding is computed on the hot path. Which integer a name receives depends on the order of insertion; it is a gauge freedom, and the machine never lets an integer comparison stand in for a semantic distinction between runs.

With names interned, every atom is a head symbol and at most four integer arguments: three for most shapes, four for cons_d and cons_s , and for inst its two channels and the index of its interned context. A state is a structure of arrays, one table per shape, one column per argument slot.

5.2 The rules as an operator

Viewing a state as a basis vector $|y\rangle$ of the free module on multisets, each rule r with premise multiset L_r and conclusion R_r defines an operator $H_r = \sum_{\theta} (\prod_{\beta \in R_r, \theta} a_{\beta}^{\dagger}) (\prod_{\alpha \in L_r, \theta} a_{\alpha})$, with $a^{\dagger}$ and a adding and removing one occurrence of a species [5, 10]. The transfer operator $T = \sum_r H_r$ represents the rules alone, independently of any term, and the reduction graph of γ is the orbit of $|y\rangle$ under T . Its matrix elements count derivations by the mass-action law: the combinators are a chemical reaction network whose reactions are generated from the shape table.

T is never materialised. It is useful because the invariants that separate subsets of the combinators by expressive power are exactly the finiteness conditions of T .

THEOREM 5.1 (THE EXPRESSIVENESS LATTICE CLASSIFIES THE OPERATOR). *Let γ be a state over a set of atoms.*

- (i) *Index set. Every name in a reduct of γ lies in the closure of the names of γ under the node shapes of the constructors present. Without constructors the reachable species are finite; with them they are the free algebra on the initial names, generated on demand.*
- (ii) *Nilpotency. Weight b_l, b_r, s at 2 and every other non-message atom at 1. Every rule except eval strictly decreases the weight. Without e the reachable operator is strictly block upper triangular, $T^{\text{wt}(\gamma)+1} = 0$, and reachability is the terminating Neumann series $(1 - T)^{-1} = \prod_i (1 + T^{2^i})$, computable by $\lceil \log_2(\text{wt}(\gamma) + 1) \rceil$ sparse squarings.*
- (iii) *Occupation. Without d and e, the combined number of messages and stores does not increase.*

Each clause is the lemma that proves an edge of the expressiveness lattice, read as a statement about T : (i) is the name-closure lemma, which makes the constructor edges strict, and (ii) is the

proof that without e the calculus is strongly normalising, hence not Turing complete. The destructor e therefore has three characterisations: it carries Turing power, it is the unique rule that breaks the grading, and it is the unique rule whose operator cannot be written schematically, since its image depends on the contents of the intern table. Every compiled input contains e in its gate, so clause (ii) applies to input-free programs and hand-written terms only; compiled programs are run from the frontier, which is what the machine below does.

5.3 Redex finding is a join

For each shape A and slot j let M_j^A be the Boolean matrix with $M_j^A[i, n] = 1$ iff the i -th occurrence of A carries name n in slot j . The whole family has one nonzero per argument, so it is the size of the term. Every rule is a conjunction of atom premises that share names, so redex finding is a relational join on these matrices. The seven two-premise rules against a message are the product $M_0^C (M_0^m)^\top$; the quote rule is $M_0^{fw} (M_0^q)^\top$; an n -ary constructor is n such products followed by a row-wise product with a mask enforcing distinct message occurrences. Two redexes *conflict* if they consume a common occurrence, so with B the redex-occurrence incidence matrix the conflict graph is the off-diagonal pattern of $BB^\top$. The whole per-step analysis is two rounds of sparse joins on term-sized operands. No matrix whose dimension is the number of states appears.

5.4 The parallel step

Definition 5.2 (Step). Redexes enabled in γ are *independent* if the occurrences they consume are disjoint. A step $\gamma \Rightarrow_S \gamma'$ fires a set S of pairwise independent redexes, removing every consumed occurrence and adding every produced atom.

LEMMA 5.3 (DIAMOND). *If r_1, r_2 are independent and enabled in γ , each remains enabled after the other fires, and both orders yield the same state up to the canonical bijection on interned names.*

PROOF. Firing removes only consumed occurrences, so disjointness preserves each premise. The produced atoms of r_i are a function of the occurrences it consumes and of the intern table, whose contributions are independent of insertion order up to the canonical bijection. Both orders give $\gamma - L_{r_1} - L_{r_2} + R_{r_1} + R_{r_2}$. $\square$

THEOREM 5.4 (STEP SOUNDNESS). *If $\gamma \Rightarrow_S \gamma'$ then $\gamma \xrightarrow{*} \gamma'$ by a sequential derivation firing exactly the members of S , in any order, and all such orders agree. Consequently a run of k steps performs $\sum_i |S_i|$ rewrites, the available speedup over a sequential interpreter is the mean step width, and k is bounded below by the causal depth of the derivation.*

PROOF. Induction on $|S|$ with Lemma 5.3. $\square$

The converse fails deliberately. Firing a *maximal* independent set excludes interleavings in which an enabled, unconflicted redex waits, and when redexes conflict, choosing which one enters S is choosing a schedule. Maximal progress is therefore a scheduler, not a semantics: the machine explores a subset of the source's behaviours, each of which is a genuine behaviour by Theorem 5.4. For the rho calculus, whose nondeterminism is precisely the resolution of races on channels, this is the expected contract, and it is the contract a tuple-space implementation offers when its cores race. We randomise priorities afresh at every step, as Luby's algorithm does [24], so that no enabled redex is starved by a fixed order.

6 IMPLEMENTATION

f1r3comb is a Rust workspace of eight crates, about 7,800 lines excluding a vendored parser. Its pipeline follows Figure 1.

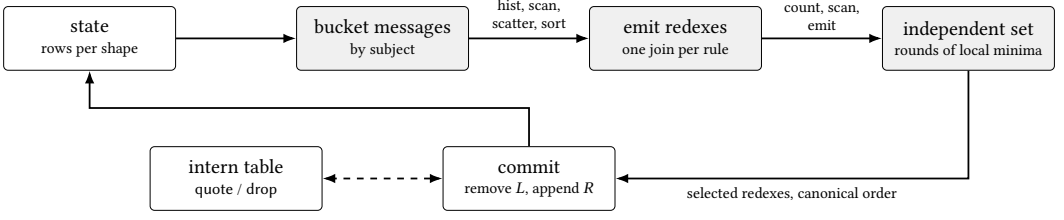


Fig. 3. One machine step. Shaded stages run on the device as generated WGSL kernels; every arrow between them is a bulk-synchronous barrier. The dashed arrow is the only access to shared mutable state, used by class (b) and (c) rules.

Front end. A parser and normaliser for the kernel of the rho calculus produce a canonical form with de Bruijn indices, so that α -equivalent programs have byte-identical encodings.

Phase one. This phase is a homomorphism on the normal form into an IR for RC^v with its own New node and a nameless encoding, so α -equivalence is equality of encodings. It classifies occurrences as in Table 1, emits the (o5) constructor chains, and posts an address request at every release site.

Phase two. This phase compiles each unit to its inst erection, with bound names written as Elias-gamma leaves beneath de Bruijn-indexed holes. Four post-passes fail compilation if violated: the single-address discipline; no address feeding an (o5) chain (Lemma 4.5); no static channel occurring as a value; and no hole outside a context. Two further schemes are available for comparison: *curried*, and *positional*, which is the unsound scheme of §4.2 kept as a negative control.

Rule table. The twenty rules are *generated* from a table of twenty-one shapes: nine base atoms, cons_i , one constructor per base shape, inst, and the curried constructor. Each rule records its consumer shape, its premises as (argument slot, message or store) pairs, and its conclusion as a class (a) atom template, a class (b) encode template, or the class (c) decode. The host machine, the GPU kernels and the manifest shipped with each GPU bundle all read this one table.

Host machine. States are structure-of-arrays tables per shape over a structural intern table. Each step sorts redexes by a canonical key, computes conclusions in that order, and interns any new names in a fixed traversal order, so runs are deterministic given a seed and host and device assign identical integers. Four resolvers are provided: maximal progress, single redex by priority, single redex uniformly at random, and an adversarial resolver that deliberately prefers redexes whose premises come from different instances.

GPU machine. The device backend uses wgpu (Vulkan, Metal or DirectX 12) and WGSL kernels generated from the rule table, which enters the shader as constant arrays: per rule, the consumer shape, the number of premises, and for each premise the argument slot it joins on and whether it reads a message or a store. A step dispatches twelve kernels (Figure 3). Redex finding is a radix-partitioned hash join: message and store rows are histogrammed by subject name, prefix-scanned, scattered and sorted within buckets; each consumer row then counts its candidate partners in the bucket of its joined slot, a second scan assigns output offsets, and an emit kernel writes redex records of eight words each. Selection is a greedy maximal independent set computed in rounds: each redex draws a 64-bit priority from a SplitMix64 finaliser keyed by the seed, the step number, its rule and the rows it consumes; each occurrence records the minimum priority among its undecided redexes by three passes of 32-bit atomic minimum (high word, low word, index); a redex that is the

minimum at every occurrence it touches is taken, and redexes touching a taken occurrence are excluded. Rounds repeat until no redex is undecided, which yields exactly the set the sequential greedy algorithm yields in priority order. The host then commits the step, sharing the host machine's commit routine, so that the intern table stays on the host. Device and host step traces are therefore identical by construction of the selection and are checked to be so by test. Moving the intern table behind a batched device-side insert-or-get is future work.

Artefacts. Every stage can be saved and fed back in: a textual IR after phase one, a canonical binary target after phase two whose header records the scheme, the constructor subset the program uses and a marking analysis, and a GPU bundle holding the deployed state, the generated kernels, and a manifest with the rule table, buffer layout, dispatch sequence and a reference trace summary that a device run must reproduce.

7 EVALUATION

We ask four questions. **RQ1:** does the compiler preserve behaviour, and do host and device agree? **RQ2:** do machine steps track causal depth rather than work on compiled rho programs, and what does compilation cost in rewrites? **RQ3:** what limits width, and can the compiler remove the limits it creates? **RQ4:** how do the erection schemes compare under concurrent instantiation?

Setup. All runs use the release build of f1r3comb under Rust 1.85 on Ubuntu 24.04, with the device backend on Mesa's software Vulkan driver (llvmpipe, LLVM 20). Step counts, widths and redex counts are properties of the machine semantics and are identical on any adapter; we do not report times.

7.1 RQ1: Correctness

The test suite comprises 48 tests in the workspace and 33 in the vendored front end, all passing. The central checks are differential against a reference reducer for the source calculus, comparing encoded observations at quiescence.

- *The regression term W* of §4.2, and a three-copy variant, agree with the source on all 100 seeds, and the crossed residue $@X!(Y)$ never appears. Under the positional negative control, W crosses on 49 of 100 seeds and the three-copy variant on 85.
- *A corpus of 19 programs* covering every occurrence kind, nested inputs, relays, (o5) chains, races, and received code with inputs run once and twice agrees with the source on 4 seeds on the host and 2 on the device, and gives identical observations at two different root addresses (address independence).
- *Adversarial scheduling.* Under the resolver that prefers cross-instance matches, W and its variants show no disagreement in 100 seeds each.
- *Host-device agreement.* Device and host step traces are identical on the regression terms and the corpus, and on every benchmark below the final state hashes agree.
- *Canonicity.* Programs differing by a permutation of parallel components compile to α -equal phase-one images and byte-equal targets.

7.2 RQ2: Width of compiled programs

The width thesis of this approach is that the available speedup is the mean number of rewrites per step, and nothing else. We therefore measure width on five families of rho programs whose causal structure is known analytically, each compiled by f1r3comb and run to quiescence under maximal progress, with three seeds (all measurements were seed-independent).

program	source comms	rewrites	parallel steps	max width	mean width	enum./fired
fan(1, 8)	8	42	20	3	2.1	1.00
fan(4, 8)	32	162	20	12	8.1	1.00
fan(16, 8)	128	642	20	48	32.1	1.00
fan(64, 8)	512	2,562	20	192	128.1	1.00
bcast(2)	3	23	9	6	2.6	1.00
bcast(4)	15	107	15	24	7.1	1.00
bcast(6)	63	443	21	96	21.1	1.00
bcast(8)	255	1,787	27	384	66.2	1.00
nested(1, 4)	4	20	20	1	1.0	1.30
nested(4, 4)	16	74	20	4	3.7	2.30
nested(16, 4)	64	290	20	16	14.5	6.30
nested(64, 4)	256	1,154	20	64	57.7	22.3
contend(4)	4	22	6	8	3.7	1.55
contend(16)	16	82	6	32	13.7	3.93
contend(64)	64	322	6	128	53.7	13.5
ship(1)	2	15	13	2	1.2	1.00
ship(4)	5	45	14	6	3.2	1.53
ship(16)	17	165	26	11	6.3	2.59
ship(64)	65	645	74	11	8.7	7.36
ship(256)	257	2,565	266	11	9.6	26.6

Table 2. Width of compiled programs under maximal progress, with the distributor of the original formulation (a left comb). *Rewrites* is the number of combinator rewrites, equal to the step count of a sequential combinator interpreter; *mean width* is rewrites per parallel step; *enum./fired* is redexes enumerated per redex fired.

fan(n, d): n independent relay chains of depth d , each link for $(y \leftarrow c_{i,j}) c_{i,j+1}!(*y)$. Causal depth d , width n .

bcast(h): a broadcast tree of height h , each node for $(y \leftarrow c_v) (c_{2v}!(*y) \mid c_{2v+1}!(*y))$. Causal depth h , width 2^{h-1} at the last level.

contend(c): c inputs and c outputs on one channel. Causal depth one, maximal contention.

nested(n, d): n clients each running d nested inputs on a private channel. Causal depth d , width n .

ship(n): higher-order. A process for $(z \leftarrow b) (*z \mid \dots \mid *z)$ with n drops receives a piece of code containing an input, runs n copies of it, and each copy serves one of n client messages. This is W at scale: every copy needs its own address and its own erection.

Table 2 gives the results. For fan, nested and contend, the number of parallel steps is independent of the width parameter across a 64-fold range, and mean width grows linearly with it: the machine's depth is the program's causal depth plus a constant for deployment and erection. For bcast, parallel steps grow as $3h + 3$ while rewrites grow as 2^h ; three machine steps per level of the tree are the output message, the distributor and the forward to the child's gate. Compilation multiplies the number of rewrites by a constant: between 4.5 and 7 combinator rewrites per source communication across the four first-order families. The machine gives this back many times over as soon as width exceeds seven.

program	chain distributor			balanced distributor		
	steps	max width	mean width	steps	max width	mean width
ship(4)	14	6	3.2	14	8	3.2
ship(16)	26	11	6.3	16	32	10.3
ship(64)	74	11	8.7	18	128	35.8
ship(256)	266	11	9.6	20	512	128.3

Table 3. The distributor's shape on the higher-order family. Rewrites are identical in both columns (Table 2).

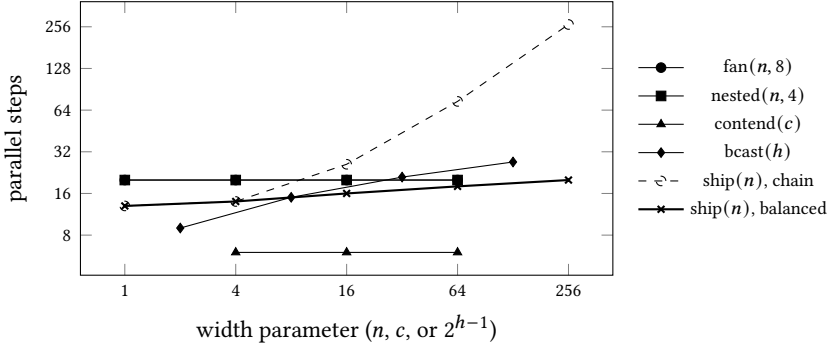


Fig. 4. Parallel steps against width. Flat lines are the goal: depth independent of the amount of parallel work. The chain distributor makes the higher-order family linear; the balanced distributor makes it logarithmic.

7.3 RQ3: What limits width, and a fix

The ship family is the exception. Its parallel steps grow linearly with n , and its mean width saturates below ten. The per-step trace locates the cause. The receiving input for $(z \leftarrow b) (*z \mid \dots \mid *z)$ has n occurrences of z , so its distributor has n atoms, and in the original formulation these form a chain: $d(x, p_0, q_1) \mid d(q_1, p_1, q_2) \mid \dots$. The chain releases one copy of the received code per step, so the n instances are erected one after another however wide the machine is. This is a depth cost the compiler introduced, not one the program has: in the source all n drops fire in the same communication.

Replacing the chain with a balanced tree of the same n atoms (§4.1) is a local change to phase one, and it leaves every conclusion of Lemma 4.2 unchanged. With it, the full test suite passes except the golden files, which record the old distributor's shape, and host and device traces still agree. Table 3 and Figure 4 show the effect. Parallel steps for ship(256) fall from 266 to 20, mean width rises from 9.6 to 128, and the number of rewrites is unchanged. The other families are unaffected, since their inputs use each bound name at most once or twice.

Contention. The last column of Table 2 measures work the machine does and discards. For contend(c) the machine enumerates c^2 candidate redexes on the shared channel and fires a perfect matching of c in one step, so the ratio grows linearly. The same effect appears for nested and, after the distributor fix, for ship: n instances request addresses at the one static channel r^* , and the n inst atoms and n addresses there form n^2 candidate pairs (77 per fired redex for ship(256)). Contention thus costs enumeration, not depth. For rules with a single message premise the enumeration is avoidable in principle: within a bucket, a matching between consumers and messages can be formed by pairing ranks after one segmented scan, and for multi-premise rules worst-case-optimal

instances	literal		curried		inst	
	runs mixed	fw. mixed	runs mixed	fw. mixed	runs mixed	fw. mixed
2	29/50	58/100	0	0	0	0
4	48/50	144/200	0	0	0	0
8	50/50	347/400	0	0	0	0
16	50/50	754/800	0	0	0	0
parallel steps to erect	5		3		2	

Table 4. Concurrent erection under maximal progress, 50 seeds. Under sequential uniform choice the literal scheme mixes in 16, 38, 49 and 50 runs respectively; the other two schemes never mix.

joins [29] apply. We have not implemented either, and until one is implemented these families would be enumeration-bound on real hardware at large c .

7.4 RQ4: Erection schemes

We compare three ways of erecting instances of a unit containing an erected forwarder and a kill, deployed as n instances at distinct fresh addresses: the literal fixed-arity erection of §4.4, the curried constructor, and inst. A forwarder is *mixed* if its two names lie beneath different addresses. Table 4 gives the results over 50 seeds from the Python reference machine that accompanies the design; the Rust host machine, whose priorities differ, reproduces them in shape (literal: 23 of 50 runs mixed at two instances, 48–50 of 50 from four; curried and inst: none). The literal scheme mixes in most runs even with two instances, and in every run from eight; maximal progress mixes more than sequential choice, because it fires every enabled constructor while leaves of several instances are present at once. Neither one-premise scheme ever mixes, under either scheduler or the adversarial resolver. inst erects in two parallel steps against the curried scheme’s three and the literal scheme’s five.

The fraction of mixed forwarders under the literal scheme approaches $1 - 1/n$, which is what uniformly random pairing of left and right leaves would give: the schedule, not the program, decides which instance each leaf joins. A device running many instances is the worst case rather than a corner case, so correctness cannot be left to the scheduler.

The curried scheme is correct wherever it applies, but it applies less often and costs more. On the recursion combinator D_x , run for 200 unfoldings, every unfolding of the hand-built image under inst takes exactly 7 parallel steps and allocates 16 new names regardless of the depth of the address, against 10 steps and 22 names under the curried scheme; sequentially, 10 rewrites against 32. The image `f1r3comb` emits for D_x also runs at 7 parallel steps per unfolding and allocates 20 names, the extra four being a separate gate pair and the posted address of the drop. On the 21 programs of the corpus and the regression terms, the curried build reaches only 12: four need a stored body with two atoms that mention the unit’s names, which requires a join, and five contain an inner input that mentions an outer bound name, which a one-hole template cannot express. The regression term W is among the former. inst compiles all 21.

The address tree has a cost of its own: names grow along every non-terminating computation, since each unfolding allocates one level deeper. With structural interning this is constant cost per unfolding, which is what the D_x measurement shows; with content hashing of full encodings it would be linear in depth.

7.5 Threats to validity

Our benchmarks are synthetic families chosen because their causal structure is known, so that the machine's depth can be checked against it; they say nothing about the width of real rho workloads, which is the measurement on which the practical value of this route depends. We report no wall-clock performance, and the device machine keeps the intern table and the commit on the host, which on real hardware would dominate the step for narrow programs. Enumeration under contention (§7.3) is quadratic in the current implementation. Theorem 4.9 is proved for enabling context labels; the minimal-label form is not established. Finally, maximal progress is a scheduler, and a program whose correctness depends on a particular interleaving being reachable under that scheduler may not see it.

8 RELATED WORK

Combinators for mobile processes. Honda and Yoshida [15] introduced concurrent combinators for the π -calculus, and Yoshida [37] developed the minimal sets and the separation results that organise them into a lattice. Both keep name restriction and eliminate only the input binder, by distributing the prefix through the body, which is where the exponential cost of Proposition 4.4 comes from. Our combinators differ in having no restriction at all, in storing continuations by quotation, and in having name constructors; the expressiveness lattice of the rho combinators differs from Yoshida's in that its expressive and observational orderings do not coincide, because a constructed name cannot be hidden [2]. The draft that first proposed name-free rho combinators [1] gave no translation from the rho calculus; §3 records why none could exist without the constructors.

Binder elimination in functional compilers. Combinatory compilation of the λ -calculus [35] removes binders at an expansion cost in nesting depth, and supercombinators and lambda lifting [16, 17] reduce that cost by abstracting each body once. Phase one is the concurrent analogue: storing the body as one quoted name is to Yoshida's elimination what supercombinators are to SKI. What has no functional analogue is phase two, because in a nondeterministic calculus the run-time instantiation of a body races with other instantiations of the same body; §4.4 is about that race.

Interaction nets on GPUs. The closest system in aim is HVM2 [34], which evaluates Lafont's interaction combinators [21, 22] on GPUs. Interaction nets are strongly confluent: every agent has one principal port, so redexes never overlap and a parallel step needs no conflict resolution. The rho calculus is deliberately not confluent — two inputs on one channel race — so the combinators must share channels, redexes overlap, and each step must choose a maximal independent set. The price is a selection phase and the enumeration cost of §7.3; the gain is a direct model of nondeterministic, communicating programs, including higher-order ones that ship code.

Multiset rewriting and nets. The target is a multiset rewriting system in the tradition of the chemical abstract machine [7] and Gamma [6], and the reflexive CHAM of the join calculus [13]. Without constructors and without e it is a place/transition net and the bulk-synchronous step is the net's step semantics [31]; adding constructors generates places on demand, and adding e lets a transition depend on the contents of a place. The independence relation of Lemma 5.3 is the usual one of trace theory [26]. The operator view of §5.2 is the Doi–Peliti representation of reaction networks [5, 10].

Relational execution. Finding redexes by joins over a term-sized incidence relation places the machine next to bottom-up Datalog engines [19] and linear-algebraic graph processing [20], and selection uses Luby's parallel independent set [24]. Moving pattern matching from run time into

the compiler plays the role flattening plays for nested data parallelism [8]: it turns irregular work into regular bulk operations over flat arrays.

Implementations and encodings of process calculi. Pict [30] and the join-calculus languages compile to sequential or thread-parallel runtimes with a central scheduler, as do tuple-space implementations of the rho calculus; parallel graph reduction machines such as GRIP [18] targeted special-purpose hardware. Our correctness criterion follows the literature on encodability and full abstraction [14, 28, 32] with context-labelled transitions [23, 33], and our relativisation to encoded contexts has the same form as for the encoding of the π -calculus in the rho calculus [3, 25].

9 CONCLUSION

A reflective process calculus can be compiled into a form a GPU accepts. The compiled form has no binders, no pattern variables and no structural congruence to decide at run time; its states are arrays of fixed-width integer rows, its redexes are found by joins, and its steps are independent sets. Reflection is what makes this cheap rather than what makes it hard: storing a continuation as a name keeps the image linear and every row fixed-width, and on the machine reflection is an intern table.

The part of the problem that needed new ideas was run-time freshness on a machine that fires everything at once. Allocating fresh apparatus beneath run-time addresses is necessary, and erecting it with fixed-arity constructors is unsound under bulk-synchronous scheduling, provably so. One single-premise combinator that instantiates a context from an address repairs it and makes erection a constant two steps. On program families with known structure, machine depth tracks causal depth, and where the compiler added depth that the program did not have, the machine's trace pointed to it and a one-line change removed it.

What remains is empirical and engineering work: measuring the width of real rho programs, moving the intern table onto the device, and removing the quadratic enumeration under contention. The operator view also has a second consumer. On the destructor-free fragment the reachability closure is a product of sparse squarings, and the modality of a spatial-behavioural logic for the combinators is a transposed matrix-vector product, so the machine that runs programs could equally check them.

DATA-AVAILABILITY STATEMENT

The compiler, host and GPU machines, test suite, and the generators and scripts for every table and figure in §7 will be submitted as an artifact. The technical reports cited as supplementary material contain full proofs.

REFERENCES

- [1] Anonymous. [n. d.]. Name-Free Combinators for Concurrency. Working draft.
- [2] Anonymous. 2026. Name-Free Combinators for the Rho Calculus: A Repair, a Linear Encoding, and the One Operation That Was Missing. Technical report, supplementary material.
- [3] Anonymous. 2026. Namespaces without a Name Server: Freshness by Communication, and a Fully Abstract Encoding of the π -Calculus in the Rho Calculus. Technical report.
- [4] Anonymous. 2026. A Spatial-Behavioural Logic for the Rho Combinators. Technical report, supplementary material.
- [5] John C. Baez and Jacob D. Biamonte. 2018. *Quantum Techniques in Stochastic Mechanics*. World Scientific.
- [6] Jean-Pierre Banâtre and Daniel Le Métayer. 1993. Programming by Multiset Transformation. *Commun. ACM* 36, 1 (1993), 98–111.
- [7] Gérard Berry and Gérard Boudol. 1992. The Chemical Abstract Machine. *Theoretical Computer Science* 96, 1 (1992), 217–248.
- [8] Guy E. Blelloch. 1996. Programming Parallel Algorithms. *Commun. ACM* 39, 3 (1996), 85–97.

- [9] Nicolaas Govert de Bruijn. 1972. Lambda Calculus Notation with Nameless Dummies. *Indagationes Mathematicae* 75, 5 (1972), 381–392.
- [10] Masao Doi. 1976. Second Quantization Representation for Classical Many-Particle System. *Journal of Physics A: Mathematical and General* 9, 9 (1976), 1465–1477.
- [11] Peter Elias. 1975. Universal Codeword Sets and Representations of the Integers. *IEEE Transactions on Information Theory* 21, 2 (1975), 194–203.
- [12] Jean-Christophe Filliâtre and Sylvain Conchon. 2006. Type-Safe Modular Hash-Consing. In *Proceedings of the 2006 Workshop on ML*. 12–19.
- [13] Cédric Fournet and Georges Gonthier. 1996. The Reflexive CHAM and the Join-Calculus. In *Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 372–385.
- [14] Daniele Gorla. 2010. Towards a Unified Approach to Encodability and Separation Results for Process Calculi. *Information and Computation* 208, 9 (2010), 1031–1053.
- [15] Kohei Honda and Nobuko Yoshida. 1994. Combinatory Representation of Mobile Processes. In *Proceedings of the 21st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 348–360.
- [16] R. John M. Hughes. 1982. Super-Combinators: A New Implementation Method for Applicative Languages. In *Proceedings of the 1982 ACM Symposium on LISP and Functional Programming*. 1–10.
- [17] Thomas Johnsson. 1985. Lambda Lifting: Transforming Programs to Recursive Equations. In *Functional Programming Languages and Computer Architecture (FPCA) (LNCS, Vol. 201)*. 190–203.
- [18] Simon L. Peyton Jones, Chris Clack, Jon Salkild, and Mark Hardie. 1987. GRIP — A High-Performance Architecture for Parallel Graph Reduction. In *Functional Programming Languages and Computer Architecture (FPCA) (LNCS, Vol. 274)*. 98–112.
- [19] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On Synthesis of Program Analyzers. In *Computer Aided Verification (CAV) (LNCS, Vol. 9780)*. 422–430.
- [20] Jeremy Kepner and John Gilbert (Eds.). 2011. *Graph Algorithms in the Language of Linear Algebra*. SIAM.
- [21] Yves Lafont. 1990. Interaction Nets. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 95–108.
- [22] Yves Lafont. 1997. Interaction Combinators. *Information and Computation* 137, 1 (1997), 69–101.
- [23] James J. Leifer and Robin Milner. 2000. Deriving Bisimulation Congruences for Reactive Systems. In *CONCUR 2000 (LNCS, Vol. 1877)*. 243–258.
- [24] Michael Luby. 1986. A Simple Parallel Algorithm for the Maximal Independent Set Problem. *SIAM J. Comput.* 15, 4 (1986), 1036–1053.
- [25] Stian Lybech. 2022. Encodability and Separation for a Reflective Higher-Order Calculus. In *Combined 29th International Workshop on Expressiveness in Concurrency and 19th Workshop on Structural Operational Semantics (EXPRESS/SOS) (EPTCS, Vol. 368)*. 95–112.
- [26] Antoni Mazurkiewicz. 1987. Trace Theory. In *Petri Nets: Applications and Relationships to Other Models of Concurrency*. LNCS, Vol. 255. Springer, 279–324.
- [27] L. Gregory Meredith and Matthias Radestock. 2005. A Reflective Higher-Order Calculus. *Electronic Notes in Theoretical Computer Science* 141, 5 (2005), 49–67.
- [28] Robin Milner. 1992. Functions as Processes. *Mathematical Structures in Computer Science* 2, 2 (1992), 119–141.
- [29] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2018. Worst-Case Optimal Join Algorithms. *J. ACM* 65, 3 (2018), 16:1–16:40.
- [30] Benjamin C. Pierce and David N. Turner. 2000. Pict: A Programming Language Based on the Pi-Calculus. In *Proof, Language, and Interaction: Essays in Honour of Robin Milner*. MIT Press, 455–494.
- [31] Wolfgang Reisig. 2013. *Understanding Petri Nets*. Springer.
- [32] Davide Sangiorgi and David Walker. 2001. *The π -Calculus: A Theory of Mobile Processes*. Cambridge University Press.
- [33] Peter Sewell. 1998. From Rewrite Rules to Bisimulation Congruences. In *CONCUR '98 (LNCS, Vol. 1466)*. 269–284.
- [34] Victor Taelin. 2024. HVM2: A Parallel Evaluator for Interaction Combinators. Technical report, Higher Order Company.
- [35] David A. Turner. 1979. A New Implementation Technique for Applicative Languages. *Software: Practice and Experience* 9, 1 (1979), 31–49.
- [36] Leslie G. Valiant. 1990. A Bridging Model for Parallel Computation. *Commun. ACM* 33, 8 (1990), 103–111.
- [37] Nobuko Yoshida. 2002. Minimality and Separation Results on Asynchronous Mobile Processes: Representability Theorems by Concurrent Combinators. *Theoretical Computer Science* 274, 1–2 (2002), 231–276.